

PaymentWebhook engineering review pack

Model revision: a2d2b60650b1

Review state: draft

Approval reviewer: none

Implementation gate: CLOSED

Named branch handoff: COMPLETE across 2 registered operations

Generated snapshot. Semantic rows come from Lean; review notes and findings come from structured review metadata.

Fast pass

Scope and changes

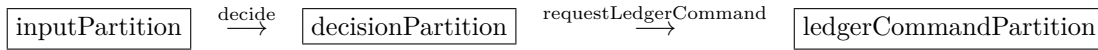
Webhook input classification, decision mapping, and ledger-command mapping in the companion example.

- First generated snapshot; no earlier projection supplied.

Boundary to challenge

PaymentWebhook.Input: Input pairs decoded WebhookPayload with a separate LedgerObservation. alreadyRecorded is not a webhook field. **Review challenge:** Is the observation from a stable transaction or snapshot? Can concurrent deliveries change it before effects occur?

Operation topology – Level 1



Partial operations: requestLedgerCommand. Arrows mean member mappings, not runtime calls. Hidden: individual members, effects, and proofs. **Operation responsibility:** PaymentWebhook.decide: payments-webhooks
PaymentWebhook.requestLedgerCommand: payments-ledger

Assumptions and open questions

- PaymentWebhook.Input: The model receives LedgerObservation.alreadyRecorded separately from the webhook payload; its acquisition and consistency are not modeled.
- PaymentWebhook.Input: What guarantees that alreadyRecorded remains valid until the handler finishes?
- PaymentWebhook.decide.duplicateSuccess: Are duplicate acknowledgments required to be idempotent under concurrent delivery?
- PaymentWebhook.decide.firstSuccess: What recovery behavior is required if recording succeeds and queue publication fails?

Deep pass

Semantic partitions

inputPartition: malformed, unsupported, failed, firstSuccess, duplicateSuccess

decisionPartition: reject, ignore, recordFailure, fulfill, acknowledgeDuplicate

ledgerCommandPartition: recordFailure, recordAndQueueFulfillment

Semantic correspondence evidence:

- PaymentWebhook.inputPartition: PaymentWebhook.inputSemanticPartition.hasMembers
- PaymentWebhook.decisionPartition: PaymentWebhook.decisionSemanticPartition.hasMembers
- PaymentWebhook.ledgerCommandPartition: PaymentWebhook.ledgerCommandSemanticPartition.hasMembers

Carrier provenance (external claims remain trusted):

Partition	Origin	Declared basis
PaymentWebhook.inputPartition	trusted-external-root	source=PaymentWebhook preclassified fixture, scope=Input pairs decoded WebhookPayload with a LedgerObservation, claim=The fixture supplies every value considered at this boundary, revision=fixture-unversioned
PaymentWebhook.decisionPartition	architecture-defined-domain	identity=PaymentWebhook.decide (architecture-defined carrier); constructors exhaust this carrier
PaymentWebhook.ledgerCommandPartition	architecture-defined-domain	identity=PaymentWebhook.requestLedgerCommand (architecture-defined carrier); constructors exhaust this carrier

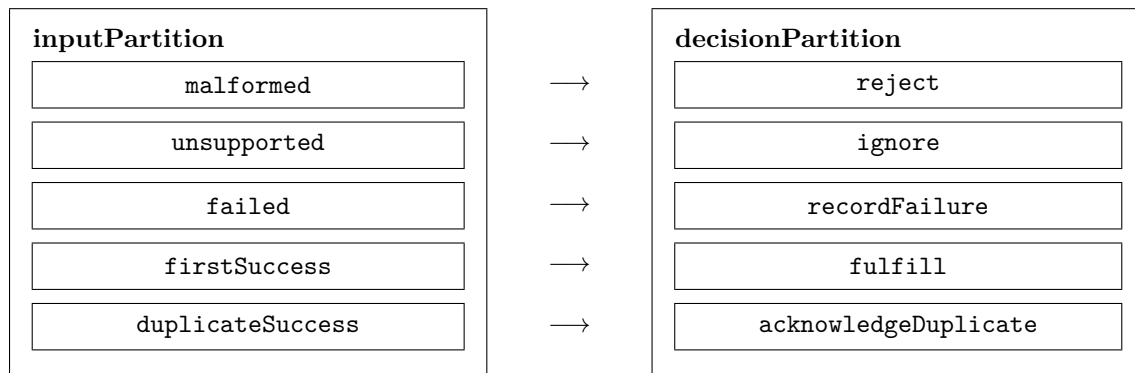
Opaque subdomains: none. No formula is available for deduction; review the meaning and consider specifying one. Coarsening: none represented in this scoped review projection.

Input member	Review meaning
malformed	eventId is empty, regardless of status or ledger observation
unsupported	eventId is present; status is neither success nor failed
failed	eventId is present; status is failed
firstSuccess	eventId is present; status is success; ledger observation is false
duplicateSuccess	eventId is present; status is success; ledger observation is true

The descriptions sit beside the Lean predicates. The proof checks predicates against the classifier, not the English wording.

decide branch map – Level 2

Focus: all declared decide branches. Hidden: predicate formulas and runtime effects.



Canonical branch	Source	Target
PaymentWebhook.decide.malformed	malformed	reject
PaymentWebhook.decide.unsupported	unsupported	ignore
PaymentWebhook.decide.failed	failed	recordFailure
PaymentWebhook.decide.firstSuccess	firstSuccess	fulfill
PaymentWebhook.decide.duplicateSuccess	duplicateSuccess	acknowledgeDuplicate

Partial ledger mapping

Decision	Ledger command
reject	$\emptyset$
ignore	$\emptyset$
recordFailure	recordFailure
fulfill	recordAndQueueFulfillment
acknowledgeDuplicate	$\emptyset$

$\emptyset$ means undefined for this member; it does not prove absence of unrelated runtime effects.

Effects and state assumptions

- `PaymentWebhook.requestLedgerCommand`: A mapping to none means no ledger-command member is requested. It does not prove the handler has no other runtime effects.
- `PaymentWebhook.decide.firstSuccess`: The companion handler records payment and then enqueues fulfillment; atomicity across those effects is not established. This example has no typed effect contract for that production sequence.

Responsibility and implementation

Branch	Responsibility	Primary code / status
PaymentWebhook.decide.malformed	payments-webhooks	archiscript:examples/payment-webhook.mjs#decide (resolved)
PaymentWebhook.decide.unsupported	payments-webhooks	archiscript:examples/payment-webhook.mjs#decide (resolved)
PaymentWebhook.decide.failed	payments-webhooks	archiscript:examples/payment-webhook.mjs#decide (resolved)
PaymentWebhook.decide.firstSuccess	payments-webhooks	archiscript:examples/payment-webhook.mjs#decide (resolved)
PaymentWebhook.decide.duplicateSuccess	payments-webhooks	archiscript:examples/payment-webhook.mjs#decide (resolved)
PaymentWebhook. requestLedgerCommand.recordFailure	payments-ledger	archiscript:examples/ payment-webhook. mjs#requestLedgerCommand (resolved)
PaymentWebhook. requestLedgerCommand.fulfill	payments-ledger	archiscript:examples/ payment-webhook. mjs#requestLedgerCommand (resolved)

Machine-checked claims

- `inputPartition` classifier agrees with `inputMembers` (`PaymentWebhook.inputSemanticPartition.hasMembers`)
- `requestLedgerCommand.comp` decide maps `firstSuccess` to `recordAndQueueFulfillment` (`PaymentWebhook.first_success_requests_fulfillment`)

- `requestLedgerCommand.comp decide` maps `duplicateSuccess` to `none` (`PaymentWebhook.duplicate__requests__no__ledger__command`)

Conditional claims: none exported in this scoped review projection. Effect and concurrency questions remain unknown. Code bindings and test references do not prove code conformance.

Unknowns and findings

- `PaymentWebhook.Input`: What guarantees that `alreadyRecorded` remains valid until the handler finishes?
- `PaymentWebhook.decide.duplicateSuccess`: Are duplicate acknowledgments required to be idempotent under concurrent delivery?
- `PaymentWebhook.decide.firstSuccess`: What recovery behavior is required if recording succeeds and queue publication fails?

RV-1 — `PaymentWebhook.Input` (open) The `alreadyRecorded` Boolean is modeled without a snapshot or transaction boundary. *Required change:* An engineer should confirm the consistency guarantee or request an explicit concurrency assumption or revised carrier.

RV-2 — `PaymentWebhook.decide.firstSuccess` (open) The model names a record-and-queue command but does not specify recovery after a successful record and failed queue publication. *Required change:* An engineer should confirm the delivery guarantee and request an effect contract or compensating behavior if required.